

□ **Exercice 1 : Spécifications**

Proposer un nom, une spécification, des préconditions et un jeu de tests pour les fonctions suivantes :

```
1. bool fonction1(int a, int b, int c)
2   {return (a==b) || (b==c) || (a==c);}
```

```
2. float fonction2(float x, float y)
2   {return 1.0f / (x*x + y*y);}
```

```
3. float fonction3(float a, float b) {
2   if (a < b)
3     {return a;}
4   else
5     {return b;}
6 }
```

□ **Exercice 2 : Fonction mystère**

On considère la fonction `mystere` suivante :

```
1 bool mystere(int n) {
2   int d = 2;
3   while (d*d <= n) {
4     if (n % d == 0)
5       {return false;}
6     d = d + 1;
7   }
8   return true;
9 }
```

1. Nommer cette fonction et en donner une spécification.
2. Tracer son graphe de flot de contrôle.
3. Proposer un jeu de tests permettant de couvrir tous les arcs.

□ **Exercice 3 : Nombre de jours dans un mois**

1. Écrire une fonction `nb_jours` qui prend en argument un entier `mois` et un entier `annee` et qui renvoie le nombre de jours de ce mois. Par exemple, `nb_jours(5, 1970)` doit renvoyer le nombre de jours du mois de mai 1970. On pourra utiliser sans la réécrire la fonction `bissextile` vue en cours.
2. Proposer des préconditions pour cette fonction.
3. Proposer un jeu de tests pour cette fonction.

□ **Exercice 4 : Triangles**

1. Écrire une fonction `triangle` qui prend en argument trois entiers strictement positifs a, b, c et renvoie :
 - 0 si les trois entiers ne sont pas les longueurs des côtés d'un triangle ;
 - 1 si les trois entiers sont les côtés d'un triangle scalène (quelconque) ;
 - 2 si les trois entiers sont les côtés d'un triangle isocèle non rectangle ;
 - 3 si les trois entiers sont les côtés d'un triangle équilatéral ;
 - 4 si les trois entiers sont les côtés d'un triangle rectangle non isocèle ;
 - 5 si les trois entiers sont les côtés d'un triangle rectangle isocèle.
2. Tracer le graphe de flot de contrôle de cette fonction.
3. Proposer un jeu de tests pour cette fonction.

□ **Exercice 5 : Compte à rebours**

On considère la fonction `C` suivante :

```

1 // Compte à rebours de n à 0 et affiche "Partez !"
2 void compte_rebours(int n)
3 {
4     while (n != 0)
5     {
6         printf("%d \n",n);
7         n = n-1;
8     }
9     printf("Partez !\n");
10 }

```

1. Cette fonction est-elle conforme à sa spécification ?
2. n est-il un variant de boucle ? Sinon, quelle propriété est manquante ?

□ **Exercice 6 :** *Maximum des éléments d'un tableau*

1. Écrire en C une fonction qui renvoie le maximum des éléments d'un tableau non vide d'entiers. Par exemple pour le tableau $\{2, 5, 4, 9, 7, 6\}$, la fonction renvoie 9.
2. Prouver la correction de cette fonction.

□ **Exercice 7 :** *Somme des éléments pairs*

1. Écrire en C une fonction qui renvoie la somme des éléments pairs d'un tableau. Par exemple pour le tableau $\{2, 5, 4, 9, 7, 6\}$, la fonction renvoie $2 + 4 + 6 = 12$.
2. Prouver la correction de cette fonction.

□ **Exercice 8 :** *Étude d'un algorithme*

On considère l'algorithme suivant :

Algorithme : Quotient dans la division euclidienne

Entrées : $a \in \mathbb{N}, b \in \mathbb{N}^*$

Sorties : $q \in \mathbb{N}$, quotient de a par b

```

1  $q \leftarrow 0$ 
2  $r \leftarrow a$ 
3 tant que  $r \geq 0$  faire
4    $q \leftarrow q + 1$ 
5    $r \leftarrow r - b$ 
6 fin
7 return  $q - 1$ 

```

1. Proposer une implémentation de cet algorithme en langage C, sous la forme d'une fonction dont on précisera soigneusement la spécification.
2. Étudier la terminaison de cet algorithme.
3. Étudier la correction de cet algorithme.

□ **Exercice 9 :** *Multiplication à la russe*

On considère l'algorithme suivant :

Algorithme : Multiplication à la russe

Entrées : $a \in \mathbb{N}, b \in \mathbb{N}$

Sorties : ab

```

1  $m \leftarrow 0$ 
2 tant que  $a > 0$  faire
3   si  $a \bmod 2 = 1$  alors
4      $m \leftarrow m + b$ 
5      $a \leftarrow a - 1$ 
6   fin
7   sinon
8      $a \leftarrow a/2$ 
9      $b \leftarrow b + b$ 
10  fin
11 fin
12 return  $m$ 

```

1. Faire fonctionner cet algorithme à la main pour $a = 7$ et $b = 5$.
2. Donner une implémentation en C de cet algorithme sous la forme d'une fonction dont on précisera soigneusement la spécification.
3. Prouver la terminaison de cet algorithme.
 ⊗ a est un variant de boucle.
4. Prouver la correction de cet algorithme.
 ⊗ $m + ab$ vaut toujours le produit des valeurs initiales de a et de b .

□ **Exercice 10 : Tri par sélection**

L'algorithme du tri par sélection d'un tableau t de longueur n consiste, pour chaque entier i de 0 à $n - 1$, à :

- rechercher le plus petit élément du sous-tableau $\{t[i], \dots, t[n - 1]\}$;
 - l'échanger avec celui situé à l'indice i .
1. Écrire cet algorithme en pseudo-code.
 2. En donner une implémentation en langage C en spécifiant soigneusement les fonctions utilisées.
 3. Proposer un jeu de tests.
 4. Prouver la correction totale de cet algorithme.

□ **Exercice 11 : Algorithme d'Euclide**

On considère l'algorithme d'Euclide par soustractions pour calculer le plus grand commun diviseur (pgcd) de deux entiers strictement positifs :

Algorithme : PGCD par soustractions successives

Entrées : $a \in \mathbb{N}^*, b \in \mathbb{N}^*$

Sorties : $\text{pgcd}(a, b)$

```

1 tant que  $a \neq b$  faire
2   si  $a > b$  alors
3     |  $a \leftarrow a - b$ 
4   fin
5   sinon
6     |  $b \leftarrow b - a$ 
7   fin
8 fin
9 return  $a$ 
```

1. Donner une implémentation en C de cet algorithme.
2. Montrer que $a + b$ est un variant de boucle et en déduire la terminaison de l'algorithme.
3. Montrer que la propriété « $\text{pgcd}(a, b) = \text{pgcd}(a_0, b_0)$ » est un invariant de boucle, où a_0 et b_0 désignent les valeurs initiales de a et b . En déduire la correction totale de l'algorithme.

□ **Exercice 12 : Recherche dichotomique**

On considère un tableau t de n entiers trié par ordre strictement croissant et une valeur cible x . On souhaite déterminer si x appartient à t :

Algorithme : Recherche dichotomique

Entrées : t tableau d'entiers de taille n trié par ordre croissant, x entier

Sorties : Indice m tel que $t[m] = x$, ou -1 si x n'appartient pas à t

```

1  $g \leftarrow 0$ 
2  $d \leftarrow n - 1$ 
3 tant que  $g \leq d$  faire
4    $m \leftarrow g + \lfloor (d - g) / 2 \rfloor$ 
5   si  $t[m] = x$  alors
6     | return  $m$ 
7   fin
8   sinon si  $t[m] < x$  alors
9     |  $g \leftarrow m + 1$ 
10  fin
11  sinon
12    |  $d \leftarrow m - 1$ 
13  fin
14 fin
15 return  $-1$ 
```

1. Proposer un jeu de tests complet en utilisant la méthode du partitionnement des données d'entrée et en testant les cas limites.
2. Montrer que la quantité $d - g + 1$ permet de prouver la terminaison de la boucle.
3. Prouver que la propriété « si x est présent dans le tableau t , alors son indice est dans $\llbracket g, d \rrbracket$ » est un invariant de boucle. En déduire la correction totale de l'algorithme.

□ **Exercice 13** : *Exponentiation rapide*

On considère l'algorithme suivant calculant x^n pour $x \in \mathbb{R}$ et $n \in \mathbb{N}$:

Algorithme : Exponentiation rapide itérative

Entrées : $x \in \mathbb{R}, n \in \mathbb{N}$

Sorties : x^n

```

1  r ← 1.0
2  p ← x
3  k ← n
4  tant que k > 0 faire
5      si k mod 2 = 1 alors
6          | r ← r × p
7      fin
8      p ← p × p
9      k ← ⌊k/2⌋
10 fin
11 return r
```

1. Faire tourner cet algorithme à la main pour $x = 3.0$ et $n = 11$ en précisant les valeurs prises par r , p et k à chaque étape.
2. Proposer une implémentation en langage C sous la forme d'une fonction `double puissance(double x, int n)` accompagnée de ses préconditions avec `assert`.
3. Prouver la terminaison de cet algorithme.
4. Prouver que la propriété I : « $r \times p^k = x^n$ » est un invariant de boucle. Conclure sur la correction de l'algorithme.

□ **Exercice 14** : *Chasse aux invariants et variants erronés*

Pour chacun des cas suivants, analyser la proposition d'un élève et expliquer pourquoi le raisonnement est incomplet ou invalide :

1. Pour la boucle `while (n > 1) { n = n / 2; }`, un élève propose $v = n$ comme variant et justifie : « n est un entier strictement décroissant à chaque tour ». Que manque-t-il pour prouver la terminaison ?
2. Pour calculer la somme $S = \sum_{k=0}^{n-1} t[k]$ avec une boucle `for (int i = 0; i < n; i++) s += t[i];`, un élève propose l'invariant I : « $s \geq 0$ ». Cet invariant permet-il de prouver la correction de la fonction ? Pourquoi ?
3. Pour la boucle `while (i < j) { i++; j--; }`, un élève propose l'invariant I : « $i = j$ ». Cet invariant est-il conservé si on démarre avec $i = 0$ et $j = 5$?